Scientific Computing WS25/26

Algorithmic Mathematics: Chapter 1

Anton Herzberg

Algorithms

An algorithm is a finite prescription with following specifications:

- **What** is an acceptable **input**?
- **Which** computational **steps** depending on input will be **performed** and in which **order**?
- **When** do algorithm **terminate** and what will be the **output**

—> Formal definitions require **concrete** computer model, programming language, at this point we will only look at examples.

- Algorithmic Mathematics deals with **design** and **analysis** of algorithms.

Formal Definitions

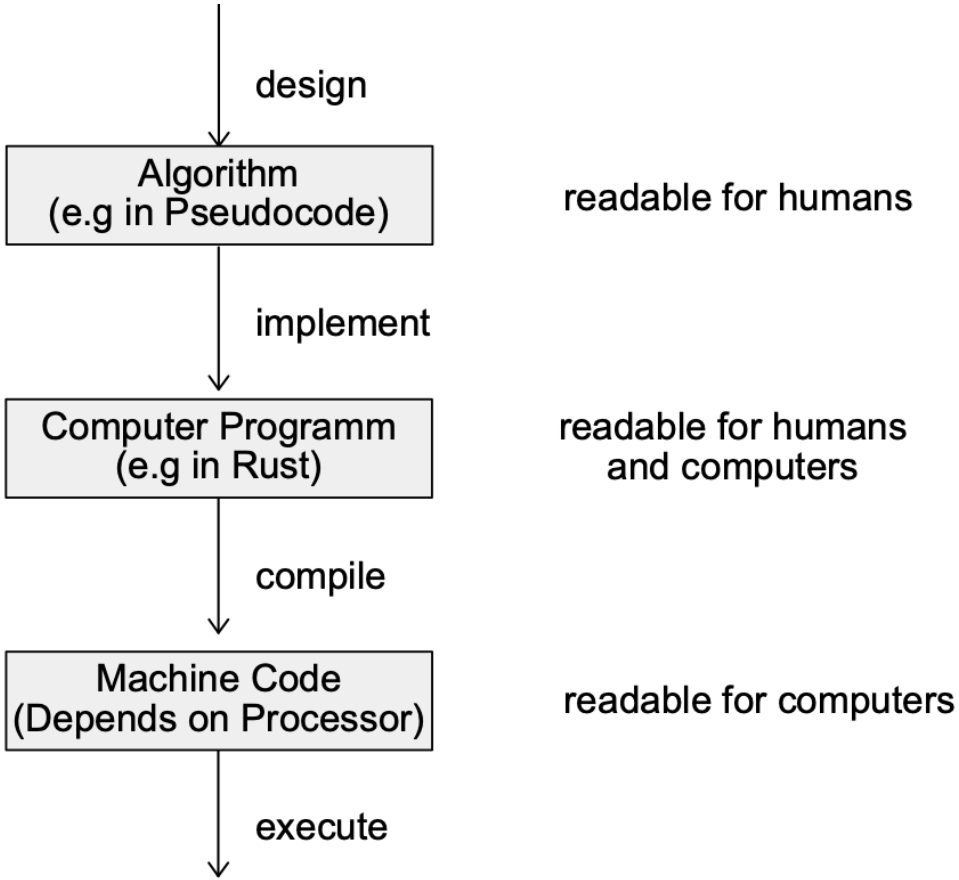
Definition 1.5 Let A be a nonempty finite set. For a given $k \in \mathbb{N} \cup \{0\}$ let A^k denote the set of functions $f: \{1, \ldots, k\} \to A$. We often write an element $f \in A^k$ as a sequence $f(1) \ldots f(k)$ and call it a **word** (or **string**) of **length** k over the **alphabet** A. The single element $\emptyset$ of A^0 is called the **empty word** (it has length 0). Put $A^* := \bigcup_{k \in \mathbb{N} \cup \{0\}} A^k$. A **language** over the alphabet A is a subset of A^*.

- We can see a computer program as a function $f: A \to B$, that turns a given input $a \in A$ into the output $f(a) \in B$

Definition 1.6 A **computational problem** is a relation $P \subseteq D \times E$ with the property that for every $d \in D$ there exists at least one $e \in E$ with $(d, e) \in P$. If $(d, e) \in P$, then e is a **correct output** for the problem P with input d. The elements of D are called the **instances** of the problem.

The computational problem P is said to be **unique** if it is a function. If D and E are languages over a finite alphabet A, then P is called a **discrete computational problem**. If D and E are subsets of $\mathbb{R}^m$ and $\mathbb{R}^n$ with $m, n \in \mathbb{N}$, respectively, then P is called a **numerical computational problem**. A unique computational problem $P: D \to E$ with $|E| = 2$ is called a **decision problem**.

- Computers only solve discrete problems
- Numeric problems needs restriction to finite input/output —> leads to rounding errors


design
Algorithm
(e.g in Pseudocode)
readable for humans
implement
Computer Programm
(e.g in Rust)
readable for humans
and computers
compile
Machine Code
(Depends on Processor)
readable for computers
execute

Algorithm 1.7 (The Square of an Integer)

Input: $x \in \mathbb{N}$.
Output: the square of x.

result $\leftarrow x \cdot x$
output result

```rust
// square.rs (Compute the Square of an Integer)
use std::io;
fn main()
{
    println!("Enter an integer: ");
    let mut input = String::new();
    io::stdin().read_line(&mut input).expect("Failed to read input");
    let x: i64 = input.trim().parse().expect("Please enter a valid integer");
    let result = x * x;
    println!("The square of {} is {}.", x, result);
}
```

Does an Algorithm always terminates?

If so, how fast?

- We consider **elementary operations** to measure the **running time**
- Basic **arithmetic** operations are **elementary**

Definition 1.13 Let $g: \mathbb{N} \to \mathbb{R}_{\geq 0}$. Then we define:

$$O(g) := \{f: \mathbb{N} \to \mathbb{R}_{\geq 0} : \exists \alpha \in \mathbb{R}_{>0}\ \exists n_0 \in \mathbb{N}\ \forall n \geq n_0 : f(n) \leq \alpha \cdot g(n)\};$$

$$\Omega(g) := \{f: \mathbb{N} \to \mathbb{R}_{\geq 0} : \exists \alpha \in \mathbb{R}_{>0}\ \exists n_0 \in \mathbb{N}\ \forall n \geq n_0 : f(n) \geq \alpha \cdot g(n)\};$$

$$\Theta(g) := O(g) \cap \Omega(g).$$

- Often f = O(g), $f \in O(g)$

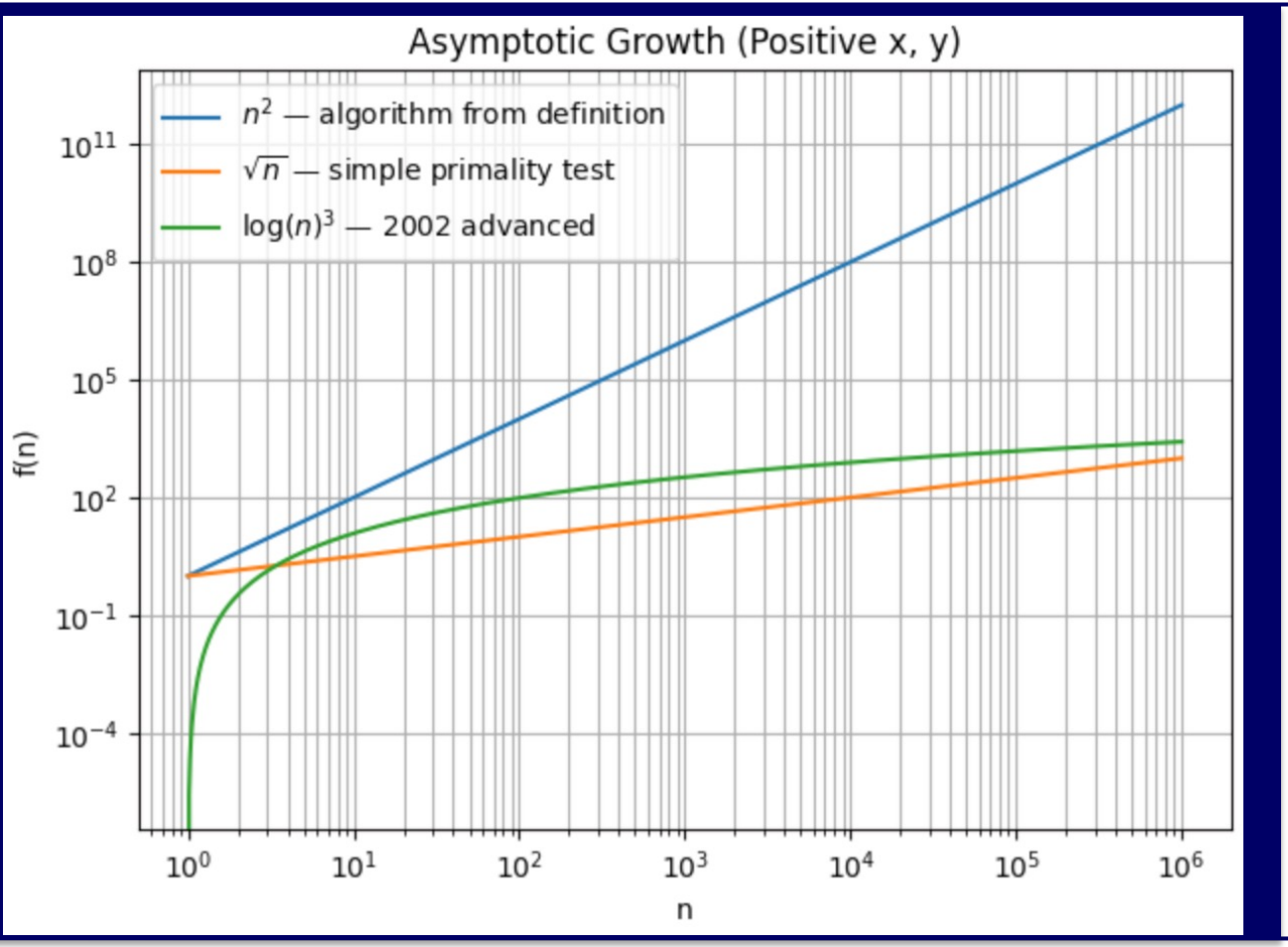
Algorithm 1.11 (Simple Primality Test)

Input: $n \in \mathbb{N}$.
Output: the answer to the question whether or not n is prime.

if $n = 1$ **then** result $\leftarrow$ "no" **else** result $\leftarrow$ "yes"
for $i \leftarrow 2$ **to** $\lfloor \sqrt{n} \rfloor$ **do**
    if i divides n **then** result $\leftarrow$ "no"
output result

- The **number of steps** is here essentially proportional to $O(\sqrt{n})$, or f(n) elementary operations and $g: n \to \sqrt{n}$, then $f = O(g)$
- 2002 Algorithm with running time $O((logn)^k)$ was found with some k
- Its a decision problem

Computational Problems: **Simple Primality Test**



Not Everything is Computable

Corollary 1.21 *The set of all C++ programs is countable.*

Theorem 1.22 *There exist functions* $f: \mathbb{N} \to \{0,1\}$ *that cannot be computed by any C++ program.*

Proof Let $\mathcal{P}$ be the (countable by Corollary 1.21) set of those C++ programs that compute a function $f: \mathbb{N} \to \{0,1\}$. Let $g: \mathcal{P} \to \mathbb{N}$ be an injective function. For $P \in \mathcal{P}$ let f^P be the function computed by P.

Consider a function $f: \mathbb{N} \to \{0,1\}$ with $f(g(P)) := 1 - f^P(g(P))$ for all $P \in \mathcal{P}$. Such an f exists because g is injective. Clearly $f \neq f^P$ for all $P \in \mathcal{P}$, hence there is no C++ program that computes f. □

- The **halting function** problem known **halting problem** can not be computed by computer
- It problem is not decidable

Algorithm 1.24 (Collatz Sequence)

Input: $n \in \mathbb{N}$.
Output: the answer to the question whether or not the Collatz sequence for n attains the value 1.

```
while n > 1
    if n mod 2 = 0
        then n ← n/2
        else n ← 3 · n + 1
output "the Collatz sequence attains the value 1"
```

- In practice **not easy** matter to **decide** wether a program **halts** or not
- We still **don't no** wether this program always **halts**
- For code look up the **Program 1.25** in Algorithmic mathematics